COMPLETE JAVA 25 TUTORIAL

A practical, example-driven handbook from beginner fundamentals to modern Java and backend-ready skills

What this guide contains

- Java setup, compiling, running programs and understanding the JVM
- Variables, types, operators, decisions, loops, arrays and methods
- Classes, objects, encapsulation, inheritance, interfaces and polymorphism
- Exceptions, strings, collections, generics, lambdas and streams
- Records, sealed classes, pattern matching, switch expressions and text blocks
- Files, dates, regular expressions, HTTP client, concurrency and virtual threads
- Modules, testing, JVM performance, secure coding and practical projects

> Version note: examples target Java SE 25. Some examples use the classic class + main method because that style remains widely used in production. Java 25 also supports compact source files and instance main methods for simpler programs.

This is a self-study handbook with representative examples, not a replacement for the full Java Language Specification or API documentation. Run each example, modify it, and consult the official documentation for API details.

Contents

1. Java and the JDK
2. Your first Java program
3. Variables, types and operators
4. Decisions and loops
5. Arrays and methods
6. Classes and objects
7. Encapsulation, inheritance and polymorphism
8. Interfaces, enums and records
9. Strings, text blocks and formatting
10. Exceptions and defensive programming
11. Collections Framework
12. Generics
13. Lambdas and functional interfaces
14. Streams and Optional
15. Modern Java: switch, pattern matching and sealed types
16. File I/O and NIO.2
17. Dates, times and regular expressions
18. Concurrency and virtual threads
19. HTTP client and JSON boundary
20. Modules, JARs and build tools
21. Unit testing with JUnit
22. JVM, memory and performance basics
23. Secure coding essentials
24. Practical projects
25. Exercises and suggested answers
26. Official references

1. Java and the JDK

Java is a statically typed, general-purpose language. Source code is usually compiled into bytecode, which runs on a Java Virtual Machine (JVM). The JVM helps the same compiled program run on different operating systems when compatible runtime support is available.

JDK, JVM and tools

- **JDK:** The development kit. It includes the compiler and tools needed to build Java applications.
- **JVM:** Executes Java bytecode and manages runtime services such as memory and garbage collection.
- **javac:** Compiles Java source files.
- **java:** Launches a Java application or, in modern JDKs, runs a source file directly.
- **jshell:** An interactive shell for trying Java expressions and statements.

Install and verify

Install a JDK 25 distribution, then open a terminal or command prompt:

```
java -version
javac -version
jshell
```

> If the command is not found, check that the JDK's bin directory is installed and available on your PATH. IDEs such as IntelliJ IDEA can also manage a JDK for a project.

2. Your first Java program

Classic form used in many production projects

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java 25!");
    }
}
```

Save as HelloWorld.java. Compile and run:

```
javac HelloWorld.java
java HelloWorld
```

Output: `Hello, Java 25!`

Compact source form

Java 25 supports compact source files for small programs and learning. This is convenient, but the classic class form is still important for reading existing code.

```
void main() {
    IO.println("Hello from a compact Java 25 program");
}
```

Save as Hello.java and run with a Java 25 launcher: `java Hello.java`. The IO helper is part of the newer simplified programming model.

Comments

```
// Single-line comment

/*
 * Multi-line comment
 */

/** Javadoc comment for APIs and classes. */
```

3. Variables, types and operators

A variable is a named storage location. Java requires a declared type, although `var` can ask the compiler to infer a local variable's type from its initializer.

```
int age = 25;
long population = 8_000_000_000L;
double price = 149.50;
boolean active = true;
char grade = 'A';
String name = "Asha";       // String is a reference type
var city = "Pune";          // inferred as String
```

Primitive types

The eight primitive types are byte, short, int, long, float, double, char and boolean. Use int for ordinary whole-number arithmetic, long for larger integers, and double for most general floating-point calculations. Use BigDecimal for many financial calculations where decimal precision matters.

Operators

```
int a = 10, b = 3;
System.out.println(a + b);  // 13
System.out.println(a / b);  // 3: integer division
System.out.println(a % b);  // 1: remainder
System.out.println(a > b);  // true

int count = 0;
count++;                    // count is now 1
```

> Use parentheses to make complex expressions clear. For money, avoid relying on binary floating-point rounding; consider BigDecimal and explicit rounding rules.

4. Decisions and loops

```
int score = 82;

if (score >= 90) {
    System.out.println("A");
} else if (score >= 75) {
    System.out.println("B");
} else {
    System.out.println("Keep practising");
}

String dayType = switch (score / 10) {
    case 10, 9 -> "Excellent";
    case 8, 7 -> "Good";
    default -> "Needs review";
};
System.out.println(dayType);
```

Loops

```
for (int i = 1; i <= 3; i++) {
    System.out.println("Round " + i);
}

int n = 3;
while (n > 0) {
    System.out.println(n);
    n--;
}

String[] names = {"Asha", "Ravi", "Meera"};
for (String person : names) {
    System.out.println(person);
}
```

Use `break` to exit a loop and `continue` to skip to the next iteration. Avoid complicated control flow when a simple loop will do.

5. Arrays and methods

```
static int add(int x, int y) {
    return x + y;
}

static int max(int[] values) {
    if (values.length == 0) {
        throw new IllegalArgumentException("Array cannot be empty");
    }
    int result = values[0];
    for (int value : values) {
        if (value > result) result = value;
    }
    return result;
}

// Example usage inside main:
int[] numbers = {4, 9, 2, 7};
System.out.println(add(3, 5));       // 8
System.out.println(max(numbers));    // 9
```

A method packages reusable behaviour. Parameters provide inputs; a return type describes the result. Java passes arguments by value—including object references, where the copied value is the reference.

Varargs

```
static int sum(int... values) {
    int total = 0;
    for (int value : values) total += value;
    return total;
}

// sum(1, 2, 3) returns 6
```

6. Classes and objects

A class defines the data and behaviour of a kind of object. An object is an instance of a class.

```
class BankAccount {
    String owner;
    double balance;

    BankAccount(String owner, double openingBalance) {
        this.owner = owner;
        this.balance = openingBalance;
    }

    void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive");
        }
        balance += amount;
    }
}

// Usage:
BankAccount account = new BankAccount("Asha", 1000);
account.deposit(250);
System.out.println(account.balance); // 1250.0
```

For a real banking application, use BigDecimal for money and protect state with private fields and validated methods; the short example is intentionally simplified.

static and instance members

An instance field belongs to each object. A static field or method belongs to the class itself. Use static for operations that do not need an object's state, not as a default for everything.

7. Encapsulation, inheritance and polymorphism

Encapsulation

```
class SafeAccount {
    private double balance;

    public SafeAccount(double openingBalance) {
        if (openingBalance < 0) throw new IllegalArgumentException();
        balance = openingBalance;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException();
        balance += amount;
    }
}
```

Private state plus public, validated operations prevents callers from putting an object into an invalid state.

Inheritance and overriding

```
class Animal {
    void speak() { System.out.println("Some sound"); }
}

class Dog extends Animal {
    @Override
    void speak() { System.out.println("Woof"); }
}

Animal pet = new Dog();
pet.speak(); // Woof: runtime polymorphism
```

Prefer composition over inheritance when one object merely needs to use another. Inheritance models an 'is-a' relationship and should be used intentionally.

8. Interfaces, enums and records

Interfaces

```
interface PaymentProcessor {
    boolean pay(int amountInPaise);
}

class DemoProcessor implements PaymentProcessor {
    @Override
    public boolean pay(int amountInPaise) {
        return amountInPaise > 0;
    }
}
```

Interfaces define contracts. They allow code to depend on an abstraction instead of a specific implementation, which improves testability and flexibility.

Enums

```
enum OrderStatus {
    NEW, PAID, SHIPPED, CANCELLED
}

OrderStatus status = OrderStatus.PAID;
if (status == OrderStatus.PAID) {
    System.out.println("Ready to process");
}
```

Records

```
record Customer(long id, String name) { }

Customer c = new Customer(101, "Asha");
System.out.println(c.name()); // Asha
```

A record is a concise way to model data. The compiler provides a constructor, accessors, equals, hashCode and toString. Records are shallowly immutable: referenced mutable objects are not automatically made immutable.

Sealed classes

```
sealed interface Shape permits Circle, Rectangle { }
record Circle(double radius) implements Shape { }
record Rectangle(double width, double height) implements Shape { }
```

A sealed type restricts which classes or interfaces may directly extend or implement it. This is useful when the set of valid alternatives should be controlled.

9. Strings, text blocks and formatting

```
String first = "Java";
String second = "25";

System.out.println(first + " " + second);
System.out.println(first.length());
System.out.println(first.equals("Java")); // compare content
System.out.println(first.substring(1));   // ava

String jsonLike = """
    {
      "language": "Java",
      "version": 25
    }
    """;
System.out.println(jsonLike);

String message = "User: %s, score: %d".formatted("Asha", 95);
System.out.println(message);
```

Strings are immutable. Use equals() for content equality; == compares references. For repeated string building in loops, consider StringBuilder.

10. Exceptions and defensive programming

```
static int divide(int a, int b) {
    if (b == 0) throw new IllegalArgumentException("b must not be zero");
    return a / b;
}

try {
    System.out.println(divide(10, 0));
} catch (IllegalArgumentException ex) {
    System.err.println("Cannot calculate: " + ex.getMessage());
}
```

Checked exceptions must be caught or declared. Runtime exceptions usually signal programming errors or invalid arguments. Do not catch Exception and silently ignore it.

Try-with-resources

```
try (var reader = java.nio.file.Files.newBufferedReader(
        java.nio.file.Path.of("input.txt"))) {
    System.out.println(reader.readLine());
} catch (java.io.IOException ex) {
    System.err.println("File operation failed: " + ex.getMessage());
}
```

Resources that implement AutoCloseable are closed automatically, even if an exception occurs.

11. Collections Framework

Arrays have fixed length. Collections provide dynamic data structures and standard operations.

```
import java.util.*;

List<String> names = new ArrayList<>();
names.add("Asha");
names.add("Ravi");
names.add("Asha");
```

```
System.out.println(names.get(0));  // Asha

Set<String> unique = new HashSet<>(names);
Map<String, Integer> scores = new HashMap<>();
scores.put("Asha", 95);
scores.put("Ravi", 88);

for (var entry : scores.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}
```

Choose the right collection

- **ArrayList:** Fast indexed access; good default for lists.
- **LinkedList:** A specialised list/deque; not automatically faster for ordinary list operations.
- **HashSet:** Unique elements; no guaranteed iteration order.
- **TreeSet:** Sorted unique elements.
- **HashMap:** Key-value lookup; no guaranteed iteration order.
- **TreeMap:** Sorted keys.
- **ArrayDeque:** Efficient stack/queue operations.

```
List<Integer> nums = new ArrayList<>(List.of(5, 1, 4, 2));
nums.sort(Integer::compareTo);
System.out.println(nums); // [1, 2, 4, 5]
```

Use immutable factory methods such as List.of() when the list should not be structurally modified. These factory-created collections reject null values.

12. Generics

Generics let classes and methods work with types while retaining compile-time type checks.

```
class Box<T> {
    private final T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
}

Box<String> box = new Box<>("hello");
String text = box.get(); // no cast required
```

Bounded type parameters

```
static <T extends Comparable<? super T>> T larger(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}

System.out.println(larger(7, 4)); // 7
```

Java generics use type erasure in most runtime representations. Primitive types cannot be used as generic type arguments; use wrappers such as Integer and Double.

13. Lambdas and functional interfaces

```
import java.util.function.*;

Predicate<String> isLong = s -> s.length() > 5;
Function<String, Integer> length = String::length;
Consumer<String> printer = System.out::println;
Supplier<String> greeting = () -> "Hello";

System.out.println(isLong.test("Microservices")); // true
System.out.println(length.apply("Java"));         // 4
printer.accept(greeting.get());
```

A functional interface has one abstract method. Lambdas provide a compact implementation. Common interfaces include Predicate, Function, Consumer, Supplier and UnaryOperator.

14. Streams and Optional

```
import java.util.*;
import java.util.stream.*;

List<String> names = List.of("Asha", "Ravi", "Ananya", "Amit");

List<String> result = names.stream()
    .filter(n -> n.startsWith("A"))
    .map(String::toUpperCase)
    .sorted()
    .toList();

System.out.println(result); // [AMIT, ANANYA, ASHA] sorted lexicographically
```

A stream describes a pipeline of operations over data. Intermediate operations such as filter and map are lazy; terminal operations such as toList, count and reduce trigger processing.

```
int total = List.of(10, 20, 30).stream()
    .reduce(0, Integer::sum);
System.out.println(total); // 60

Optional<String> maybeName = Optional.of("Asha");
String display = maybeName.orElse("Guest");
```

Do not use streams for every task. A normal loop is often clearer for complex branching, mutation or debugging. Avoid calling get() on Optional without checking presence.

15. Modern Java: switch, pattern matching and sealed types

Switch expressions

```
int month = 4;
String season = switch (month) {
    case 12, 1, 2 -> "Winter";
    case 3, 4, 5 -> "Spring";
    case 6, 7, 8 -> "Summer";
    case 9, 10, 11 -> "Autumn";
    default -> throw new IllegalArgumentException("Invalid month");
};
```

Pattern matching for instanceof

```
Object value = "hello";
if (value instanceof String text && !text.isBlank()) {
    System.out.println(text.toUpperCase());
}
```

Pattern matching with switch

```
static String describe(Object value) {
    return switch (value) {
        case null -> "null value";
        case Integer i -> "integer: " + i;
        case String s -> "text: " + s;
        default -> "other";
    };
}
```

Pattern matching combines a type test with a variable binding. Java 25 includes modern pattern-matching capabilities; check the official language updates for the exact feature status and syntax of each release.

16. File I/O and NIO.2

```
import java.nio.file.*;
import java.io.IOException;
import java.util.List;

Path path = Path.of("notes.txt");
try {
    Files.write(path, List.of("Java 25", "Practice every day"));
    List<String> lines = Files.readAllLines(path);
    lines.forEach(System.out::println);
} catch (IOException ex) {
    System.err.println("Could not access file: " + ex.getMessage());
}
```

Path represents a file-system path. Files offers convenient static methods. For very large files, prefer streaming APIs rather than loading the entire file into memory.

Directory listing

```
try (var paths = Files.list(Path.of("."))) {
    paths.filter(Files::isRegularFile)
         .map(p -> p.getFileName().toString())
         .sorted()
         .forEach(System.out::println);
} catch (IOException ex) {
    ex.printStackTrace();
}
```

17. Dates, times and regular expressions

```
import java.time.*;
import java.time.format.DateTimeFormatter;

LocalDate today = LocalDate.now();
LocalDate dueDate = today.plusDays(14);
LocalDateTime meeting = LocalDateTime.now();
Instant timestamp = Instant.now();

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy");
System.out.println(today.format(formatter));
System.out.println("Due: " + dueDate);
```

Use java.time instead of legacy Date/Calendar for most new code. Instant represents a point on the UTC timeline; LocalDate represents a date without a time zone; ZonedDateTime includes a zone.

```
import java.util.regex.Pattern;

boolean valid = Pattern.matches("[A-Z]{2}[0-9]{4}", "AB1234");
System.out.println(valid); // true
```

Regular expressions are useful for text matching, but they are not a substitute for complete validation rules or a parser for complex structured formats.

18. Concurrency and virtual threads

Concurrency lets tasks make progress independently. Parallelism means tasks execute at the same time. Threads share process memory, so shared mutable state must be handled carefully.

ExecutorService

```
import java.util.concurrent.*;

try (ExecutorService executor = Executors.newFixedThreadPool(2)) {
    Future<Integer> future = executor.submit(() -> 20 + 22);
    System.out.println(future.get()); // 42
} catch (InterruptedException ex) {
    Thread.currentThread().interrupt();
} catch (ExecutionException ex) {
    throw new RuntimeException(ex.getCause());
}
```

Virtual threads

```
Thread worker = Thread.ofVirtual().start(() ->
    System.out.println("Running in a virtual thread")
);
worker.join();
```

Virtual threads are lightweight threads designed to make thread-per-task code scale well for many blocking I/O workloads. They do not make CPU-bound computation faster by themselves.

Race conditions

```
import java.util.concurrent.atomic.AtomicInteger;

AtomicInteger counter = new AtomicInteger();
Runnable task = () -> counter.incrementAndGet();
```

Use atomic types, locks, concurrent collections or immutability where appropriate. The best solution is often to avoid shared mutable state.

19. HTTP client and JSON boundary

```
import java.net.URI;
import java.net.http.*;
import java.time.Duration;

HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(5))
    .build();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://example.com/"))
    .timeout(Duration.ofSeconds(10))
    .GET()
    .build();

try {
    HttpResponse<String> response =
        client.send(request, HttpResponse.BodyHandlers.ofString());
    System.out.println("Status: " + response.statusCode());
    System.out.println(response.body());
} catch (java.io.IOException ex) {
    System.err.println("Network error: " + ex.getMessage());
} catch (InterruptedException ex) {
    Thread.currentThread().interrupt();
}
```

Use HTTPS for sensitive data, set timeouts, handle non-2xx status codes, and do not log credentials or personal data. The JDK has an HTTP client, but JSON serialization typically uses a library such as Jackson or JSON-B.

20. Modules, JARs and build tools

Packages organize classes and avoid naming collisions. Modules add an explicit boundary for packages and dependencies.

```
module com.example.app {
    exports com.example.api;
    requires java.net.http;
}
```

A modular project typically has a module-info.java file at its source root. Small projects can begin without modules.

Build with javac and package a JAR

```
javac -d out src/com/example/Main.java
jar --create --file app.jar --main-class com.example.Main -C out .
java -jar app.jar
```

For production projects, Maven or Gradle manages dependencies, compilation, tests and packaging. Commit the build file and wrapper scripts where appropriate so builds are reproducible.

21. Unit testing with JUnit

Unit tests check small units of behaviour automatically. The following example shows the shape of a JUnit 5 test; add the JUnit Jupiter dependency to your Maven or Gradle build.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    @Test
    void addsTwoNumbers() {
        assertEquals(8, 3 + 5);
    }

    @Test
    void divisionByZeroIsRejected() {
        assertThrows(IllegalArgumentException.class,
            () -> {
                int divisor = 0;
                if (divisor == 0) throw new IllegalArgumentException();
            });
    }
}
```

Good tests are repeatable, isolated and named for the behaviour they verify. Include normal cases, boundary cases and invalid input. Test behaviour, not private implementation details.

22. JVM, memory and performance basics

- **Heap:** Stores most objects; garbage collection reclaims objects that are no longer reachable.
- **Stack:** Each thread has stack frames for method calls and local execution state.
- **Metaspace:** Holds class metadata in the HotSpot JVM.
- **JIT compilation:** Frequently executed code may be compiled to native machine code at runtime.
- **Garbage collection:** Reclaims unreachable objects; it does not fix memory leaks caused by objects remaining reachable.
- **Profiling:** Measure before optimizing. Look at CPU, allocations, latency and I/O.

```
java -XX:+PrintCommandLineFlags -version
jcmd <pid> VM.version
jcmd <pid> GC.heap_info
jfr start name=profile settings=profile duration=60s filename=profile.jfr
```

Replace `<pid>` with the process ID. Available jcmd commands and flags depend on the JDK build and environment. Use Java Flight Recorder and a profiler to investigate actual bottlenecks rather than guessing.

23. Secure coding essentials

- Validate untrusted input at system boundaries and use allow-lists where practical.
- Use prepared statements or parameterized database APIs; never concatenate raw user input into SQL.
- Keep secrets out of source control, logs, URLs and exception messages.
- Use TLS for network traffic and verify certificates correctly.
- Apply least privilege to files, services and cloud credentials.
- Keep dependencies updated and scan them for known vulnerabilities.
- Avoid deserializing untrusted native Java serialization data.
- Handle errors without exposing stack traces or sensitive internal details to end users.

Parameterized SQL idea

```
String sql = "SELECT id, name FROM customer WHERE email = ?";
try (var connection = dataSource.getConnection();
     var statement = connection.prepareStatement(sql)) {
    statement.setString(1, email);
    try (var result = statement.executeQuery()) {
        while (result.next()) {
            System.out.println(result.getLong("id"));
        }
    }
}
```

This example assumes a configured DataSource and an email variable. Parameterized statements separate SQL structure from user-supplied values.

24. Practical projects

Project 1: Expense tracker (beginner)

- Create Expense with id, date, category and amount.
- Store expenses in a List first; later persist them to a file or database.
- Implement add, list, category filter and total-by-month features.
- Validate amount and date inputs; write unit tests.

Project 2: Library management (intermediate)

- Model Book, Member and Loan.
- Use Map for lookup by ID and Set/List where appropriate.
- Prevent borrowing an unavailable book.
- Add repository and service layers, then persistence with JDBC or Spring Data.

Project 3: Order service (advanced)

- Build a Spring Boot REST API for orders.
- Validate requests and return suitable HTTP status codes.
- Persist orders in PostgreSQL and write integration tests.
- Publish order events to Kafka and make consumers idempotent.
- Add logging, metrics, security, Docker and a cloud deployment pipeline.

Keep the first version small and working. Add complexity in steps; a finished modest project is more valuable than a huge unfinished one.

25. Exercises and suggested answers

Exercise 1: Find the even numbers

Given a list of integers, print only even numbers.

```
List<Integer> values = List.of(1, 2, 3, 4, 5, 6);
values.stream()
      .filter(n -> n % 2 == 0)
      .forEach(System.out::println);
```

Exercise 2: Count word occurrences

```
String sentence = "java is fun and java is useful";
Map<String, Long> counts = Arrays.stream(sentence.split("\\s+"))
    .collect(java.util.stream.Collectors.groupingBy(
        word -> word, java.util.stream.Collectors.counting()));
System.out.println(counts);
```

Exercise 3: Model a product with a record

```
record Product(String sku, String name, int priceInPaise) {
    Product {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU required");
        }
        if (priceInPaise < 0) {
            throw new IllegalArgumentException("Price cannot be negative");
        }
    }
}
```

Exercise 4: Choose a collection

You need unique email addresses and fast average membership checks. A HashSet is a reasonable starting choice. If insertion order must be preserved, consider LinkedHashSet.

Exercise 5: Explain the difference

ArrayList vs HashSet: List preserves positional order and permits duplicates; Set models uniqueness. HashSet does not guarantee iteration order.

26. Official references

Dev.java — updated Java learning tutorials

https://dev.java/learn/

Java SE 25 documentation

https://docs.oracle.com/en/java/javase/25/

Java SE 25 API documentation

https://docs.oracle.com/en/java/javase/25/docs/api/

Java Language Specification, Java SE 25

https://docs.oracle.com/javase/specs/jls/se25/jls25.pdf

Java language basics

https://dev.java/learn/language-basics/

IntelliJ IDEA: create and run a Java 25 application

https://www.jetbrains.com/help/idea/creating-and-running-your-first-java-application.html

JUnit 5 user guide

https://junit.org/junit5/docs/current/user-guide/

Suggested study plan

- **Week 1:** Setup, syntax, types, decisions, loops, arrays and methods.
- **Week 2:** OOP, interfaces, inheritance, exceptions and strings.
- **Week 3:** Collections, generics, lambdas and streams.
- **Week 4:** Modern Java, file I/O, date/time, HTTP and concurrency.
- **Week 5:** Testing, JVM basics, secure coding and a practical project.
- **Week 6:** Build a backend service and revise with coding exercises.

Best way to learn: type every example yourself, predict the output before running it, deliberately change inputs, and write tests. When comfortable with Core Java, move to Spring Boot, REST APIs, databases, Kafka and cloud deployment.